



UNIDADE I

Programação Orientada a Objetos Com C#

Prof. MSc. Tarcísio Peres

Conteúdo da Disciplina

- Fundamentos de C#.
- POO – pilares e encapsulamento na prática.
- Abstração e interfaces.
- Herança e polimorfismo.
- Coleções e genéricos.
- LINQ essencial.
- Erros e testes.
- Memória e concorrência introdutória.

Conteúdo da Unidade I

- Sintaxe C#.
- Tipos.
- Nullability.
- Var.
- Organização de solução/projeto.
- Visual Studio 2026.
- dotnet CLI (estrutura .csproj, SDK-style).

Sintaxe Familiar e Estruturada do C#

- A linguagem C# pertence à família das linguagens derivadas de C, possuindo uma sintaxe bastante familiar para quem já utilizou C, C++, Java ou JavaScript. Assim como nessas linguagens, em C#, cada instrução termina com ponto e vírgula (;) e blocos de código são delimitados por chaves { e }.
- Os identificadores (nomes de variáveis, métodos, classes etc.) são case-sensitive, ou seja, diferenciam maiúsculas de minúsculas – por exemplo, Aluno e aluno seriam dois identificadores distintos.
- C# inclui as estruturas de controle usuais, como if/else para condicionais, switch para seleção de casos e laços de repetição for, while e foreach, com sintaxes muito semelhantes às do C/C++.
- A linguagem também suporta comentários de linha única (iniciados com //) e de múltiplas linhas (delimitados por /* e */). Comentários são ignorados pelo compilador e servem apenas para melhorar a compreensão do código por parte dos desenvolvedores, permitindo documentar trechos de código ou adicionar explicações detalhadas.

"Olá, Mundo!"

Estilo tradicional:

- Começamos declarando um namespace para o programa.

Saída no console: É o texto que um programa envia para a tela do terminal - a janela de letras na qual digitamos comandos. Em sistemas modernos, esse terminal pode ser o Prompt de Comando ou PowerShell (Windows), o aplicativo Terminal (macOS e Linux), ou o terminal integrado de IDEs como Visual Studio Code. Tecnicamente, essa saída é o fluxo de saída padrão (standard output, "stdout").

Para programas C# executáveis – isto é, a primeira função a ser invocada quando o programa é iniciado.

```
using System;
namespace Exemplo {
    class Program {
        static void Main() {
            // Imprime "Olá, Mundo!" no console
            Console.WriteLine("Olá, Mundo!");
        }
    }
}
```

"Olá, Mundo!"

Estilo moderno (C# 9+):

- Em versões modernas do C#, é possível escrever um programa completo utilizando instruções de nível superior (top-level statements), sem a necessidade explícita de declarar uma classe Program ou um método Main.
- Este estilo minimalista é perfeito para programas simples e scripts rápidos.
- Graças às using directives implícitas habilitadas pelos modelos de projeto atuais, não foi necessário escrever using System; no início do arquivo – por padrão, o namespace básico System e alguns outros já estão disponíveis no escopo global.

// Programa "Olá, Mundo" utilizando instruções de nível superior em C#

```
Console.WriteLine("Olá, Mundo!");
```

Sistema de Tipos Forte e Seguro

A linguagem e a plataforma .NET fornecem diversos tipos de dados fundamentais (built-in), que incluem tipos numéricos, caractere, booleano e outros. Eles residem em namespaces do .NET (principalmente em System), mas possuem palavras-chave em C# que os representam:

- int corresponde ao tipo inteiro de 32 bits assinado (System.Int32);
- double é um tipo de ponto flutuante de 64 bits (System.Double);
- char representa um caractere Unicode de 16 bits (System.Char);
- bool um valor booleano lógico (System.Boolean, que pode ser true ou false).
- C# é fortemente tipada: cada variável e constante possui um tipo bem definido em tempo de compilação. Toda expressão avalia para um tipo específico, conhecido pelo compilador.
- Detecção precoce: operações envolvendo valores, geralmente, só são permitidas se os operandos forem de tipos compatíveis.
- A checagem em tempo de compilação ajuda a evitar uma classe de erros comuns e permite corrigir problemas antes mesmo de executar o programa, promovendo maior segurança e robustez no código.

Tipos de Dados Fundamentais em C#

- C# faz distinção entre tipos por valor e tipos por referência.
- Essa distinção afeta como os dados são armazenados na memória e como são passados nas atribuições ou chamadas de métodos.

Categoria	Detalhamento
Inteiros	sbyte, byte, short, ushort, int, uint, long, ulong - variando em tamanho (8 a 64 bits) e sinal;
Ponto flutuante	float (32 bits, precisão simples) e double (64 bits, precisão dupla) para números em ponto flutuante; além de decimal (128 bits) para precisão decimal alta (usado em finanças, por exemplo);
Caractere	char - representa um único caractere Unicode;
Booleano	bool - representa um valor lógico verdadeiro ou falso;
Texto	string - uma sequência de caracteres Unicode (embora string seja tecnicamente um objeto de referência, não um tipo por valor);
Objetos base	object - o tipo base de todos os outros tipos em C#, equivalente a System.Object (todas as classes e tipos por valor derivam, direta ou indiretamente, de object).

Tipos por Valor

- Tipos por valor armazenam diretamente seus dados. Uma variável de um tipo por valor contém em si o valor atribuído, e não uma referência a um valor em outro lugar.
 - Tipos numéricos primitivos (`int`, `double`, etc.), `bool`, `char` e `struct` (estruturas);
 - Alocação: por padrão na stack (pilha) de execução ou embutidos dentro de estruturas.
 - Ao atribuir uma variável de tipo por valor a outra, realiza-se uma cópia dos dados. Duas variáveis mantêm valores independentes.
 - Não admitem valor nulo por padrão, a menos que declarados como `nullable` (ex: `int?`).

```
int x = 5;  
int y = x; // cópia do valor  
y = 10; // x continua 5
```

Tipos por Referência

- Tipos por referência armazenam referências (endereços) para objetos alocados no heap (área de memória dinâmica).
 - Classes (class), arrays (vetores), strings (cadeia de caracteres), interfaces, delegates.
 - Alocação: os objetos são criados no heap com o uso do operador new.
 - Ao atribuir uma variável de referência a outra, não copiamos os dados do objeto, mas sim a referência. Ambas variáveis apontam para o mesmo objeto.
 - Podem representar a ausência de um valor real através de null.

```
var obj1 = new Pessoa();  
var obj2 = obj1; // mesma referência  
obj2.Nome = "Ana";  
// obj1.Nome também é "Ana"
```

Entendendo a Stack na Memória

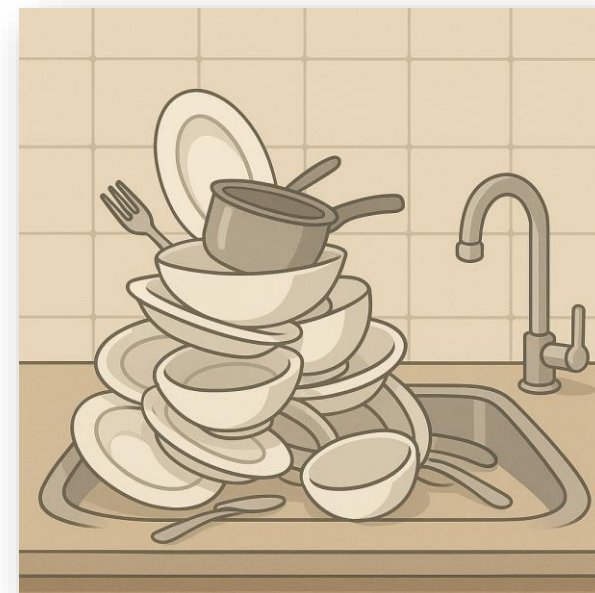
- A stack funciona como uma pilha de pratos: tudo que entra por cima sai por cima, seguindo a ordem last-in, first-out (LIFO). Serve de suporte imediato à execução de funções.
- Quando um procedimento começa, o sistema cria um quadro de ativação que reúne parâmetros, variáveis locais e o endereço de retorno. Ao término, esse quadro é descartado integralmente.
- Vantagens: Gestão automática e baratíssima – basta mover o ponteiro da pilha. Muito rápido, previsível e favorece o uso de cache.
- Limitações: Espaço limitado, vidas curtas e bem delimitadas. Empilhar estruturas grandes ou encadear chamadas profundas pode causar estouro de pilha (stack overflow).



Fonte: Imagem produzida pelo autor com a ferramenta de inteligência artificial generativa da OpenIA.

Entendendo Stack e Heap na Memória

- O heap lembra um almoxarifado: há espaço para itens de tamanhos variados, guardados e recuperados conforme a necessidade, sem seguir rigidez de ordem.
- Atende a dados cujo tamanho ou duração não cabem nas amarras da pilha: estruturas dinâmicas, coleções que crescem, objetos que atravessam fronteiras de funções.
- Em ambientes com coletor de lixo como .NET, o *runtime* monitora a alcançabilidade dos objetos e libera a memória quando deixam de ser referenciados – processo automático.
- Custo: A alocação exige metadados e coordenação do alocador. Referências mantidas desnecessariamente podem causar retenção de memória. Coletores mal configurados podem introduzir pausas perceptíveis.



Fonte: Imagem produzida pelo autor com a ferramenta de inteligência artificial generativa da OpenIA.

Tipos Anuláveis: Lidando com Ausência de Valor

- Embora tipos por valor não possam ser nulos intrinsecamente, o C# introduziu os tipos anuláveis por valor (nullable value types).
- Usando a sintaxe de interrogação após o tipo – por exemplo, `int?` em vez de `int` –, podemos declarar um tipo por valor que aceite representar a ausência de valor através de `null`.
- Em termos técnicos, `int?` é um complemento sintático para `System.Nullable<int>`, uma estrutura genérica que encapsula um valor do tipo especificado e um flag indicando se há um valor presente ou não.
- Esse recurso foi criado para lidar com situações como campos de banco de dados ou informações opcionais que possam ser "desconhecidas" ou "indisponíveis".

```
int? idade = null; // OK - int? pode ser nulo
if (idade.HasValue) {
    Console.WriteLine($"Idade: {idade.Value}");
} else { Console.WriteLine("Idade não
informada"); }
```

Declaração de Variáveis e Inferência de Tipo

- Na declaração explícita, especificamos o tipo, seguido do nome da variável e, opcionalmente, uma atribuição inicial.

```
int quantidade = 5; // quantidade é um inteiro (int) inicializado com 5
double preco = 3.99; // preco é um double (ponto flutuante) inicializado com 3.99
char inicial = 'A'; // inicial é um caractere com valor 'A'
bool ativo = true; // ativo é um booleano com valor verdadeiro
string saudacao = "Bom dia"; // saudacao é uma string com o texto "Bom dia"
```

- Desde C# 3.0, usamos var para que o compilador infira o tipo da expressão de inicialização.

```
var i = 5; // i é inferido como int (System.Int32)
var mensagem = "Olá"; // mensagem é inferido como string(System.String)
var valores = new int[] { 1, 2, 3 }; // valores é inferido como int[] (array de int)
var cliente = new Cliente(); // cliente é inferido como Cliente (tipo definido pelo usuário)
```


Organização: Projetos e Soluções no .NET

- O código-fonte é escrito dentro de projetos
- A solução (arquivo `.sln`) é um contêiner que agrega projetos, registra dependências entre eles, as configurações de compilação e execução
- Cada projeto (arquivo `.csproj`) pode ser um aplicativo de console, um

01

Solução (.sln)

Contêiner que agrega projetos, registra dependências e guarda configurações de compilação/execução

Diretórios Lógicos:

- `src/` – código de produção
- `tests/` – testes automatizados
- `samples/` – exemplos de uso
- `build/` – scripts de CI/CD
- `docs/` – documentação
- nomes no formato `Empresa.Produto.Módulo`

```
MinhaSolucao/  
  src/  
    MinhaEmpresa.Produto.Core/ # biblioteca de domínio (class library)  
    MinhaEmpresa.Produto.Infrastructure/ # acesso a dados/integrações  
    MinhaEmpresa.Produto.Api/ # Web API (ASP.NET Core)  
    MinhaEmpresa.Produto.Cli/ # app de console (opcional)  
  tests/  
    MinhaEmpresa.Produto.Core.Tests/ # testes da camada Core  
    MinhaEmpresa.Produto.Infrastructure.Tests/ # testes da  
Infrastructure  
  MinhaEmpresa.Produto.Api.Tests/ # testes de integração/endpoint  
  samples/  
    MinhaEmpresa.Produto.Samples/ # exemplos de uso de APIs públicas  
  build/ # scripts de build/pipeline  
  docs/ # documentação  
  Directory.Build.props # propriedades compartilhadas  
  Directory.Build.targets # alvos de build compartilhados  
  Directory.Packages.props # gerenciamento central de pacotes (NuGet)  
MinhaSolucao.sln
```

Anatomia de um Arquivo .csproj Moderno

- Nos projetos criados a partir dos modelos atuais, os arquivos de projeto adotam o formato baseado no SDK (`<Project Sdk="Microsoft.NET.Sdk">`).
- Um .csproj mínimo concentra propriedades essenciais – versão da linguagem, nulabilidade e usings implícitos – tornando as regras de compilação explícitas e reproduzíveis.
- A colaboração entre módulos da mesma solução ocorre por referências de projeto, que mantêm o grafo de dependências explícito. `<ProjectReference>` referencia outros projetos.
- A obtenção de bibliotecas externas usa pacotes NuGet. O gerenciamento centralizado de versões no arquivo `Directory.Packages.props` garante coerência.

Quando uma biblioteca precisa atender a mais de uma versão do .NET, usamos:

- `TargetFrameworks` (plural);
- Políticas transversais à solução ficam em arquivos compartilhados: `Directory.Build.props` estabelece padrões comuns, `Directory.Build.targets` permite acoplar etapas de build, e `.editorconfig` define estilo de código.

Interatividade

No C# 14, ao atribuir uma variável de um tipo declarado com `class` para outra variável do mesmo tipo, o que a atribuição copia?

- a) Ela copia o objeto inteiro para um novo endereço de memória.
- b) Ela copia a referência para o mesmo objeto.
- c) Ela copia apenas os campos públicos e ignora os campos privados.
- d) Ela copia apenas os membros imutáveis e recria os membros mutáveis.
- e) Ela não permite a atribuição para tipos declarados com `class`.

Resposta

No C# 14, ao atribuir uma variável de um tipo declarado com `class` para outra variável do mesmo tipo, o que a atribuição copia?

- a) Ela copia o objeto inteiro para um novo endereço de memória.
- b) Ela copia a referência para o mesmo objeto.**
- c) Ela copia apenas os campos públicos e ignora os campos privados.
- d) Ela copia apenas os membros imutáveis e recria os membros mutáveis.
- e) Ela não permite a atribuição para tipos declarados com `class`.

Visual Studio 2026

- O Visual Studio 2026 é a versão mais recente da IDE da Microsoft, lançada em novembro de 2025 e identificada internamente como versão 18, com foco em integração profunda de inteligência artificial e no suporte nativo a .NET 10 e C# 14, além de apresentar melhorias de desempenho em relação ao Visual Studio 2022.
- Em vez de um único produto, ele aparece como uma família de versões, que combinam basicamente o mesmo núcleo técnico, mas com diferenças de público-alvo, direitos de uso e conjunto de ferramentas adicionais.
- A edição Visual Studio Community 2026 é a porta de entrada gratuita. A própria Microsoft a descreve como uma IDE “free, fully-featured” para estudantes, contribuidores de projetos open source e desenvolvedores individuais.

Visual Studio 2026

- Em termos práticos, isso significa que o conjunto de recursos centrais de edição de código, depuração, testes, integração com Git e suporte a .NET 10, C# 14, C++ e demais workloads modernos é o mesmo da linha paga, com restrições concentradas no tipo de organização e no número de usuários que podem utilizar tal versão em cenários comerciais: organizações não empresariais, com até cinco usuários, podem adotar o Community de forma legítima, enquanto grandes empresas precisam migrar para o Professional ou o Enterprise, conforme as licenças oficiais.
- Git é um sistema de controle de versão distribuído usado para acompanhar o histórico de alterações em arquivos, especialmente código-fonte.
- Ele permite trabalhar com branches, mesclar contribuições de vários desenvolvedores e manter um repositório completo em cada máquina, com sincronização posterior com servidores remotos.

Visual Studio 2026

- Acima da edição Community está o Visual Studio Professional 2026, pensado para equipes pequenas e ambientes corporativos em que já existe a necessidade de licenciamento comercial formal, suporte e benefícios de assinatura.
- Em termos de experiência, ele compartilha praticamente todos os recursos de desenvolvimento da edição Community, mas vem integrado com vantagens ligadas a assinaturas (acesso a serviços, suporte, benefícios de nuvem em alguns planos) e é o caminho típico para empresas menores que precisam cumprir requisitos de compliance e gestão de licenças sem necessariamente demandar todos os recursos avançados.
- Já o Visual Studio Enterprise 2026 é voltado a organizações de qualquer porte que precisam de recursos avançados de arquitetura, depuração, testes e governança.
- Em outras palavras, é a edição que agrega, por exemplo, ferramentas mais sofisticadas de diagnóstico e análise de desempenho, recursos de teste automatizado em escala e integrações pensadas para cenários de grandes equipes, múltiplos serviços e ambientes complexos, além de apresentar benefícios adicionais de nuvem.

Visual Studio 2026

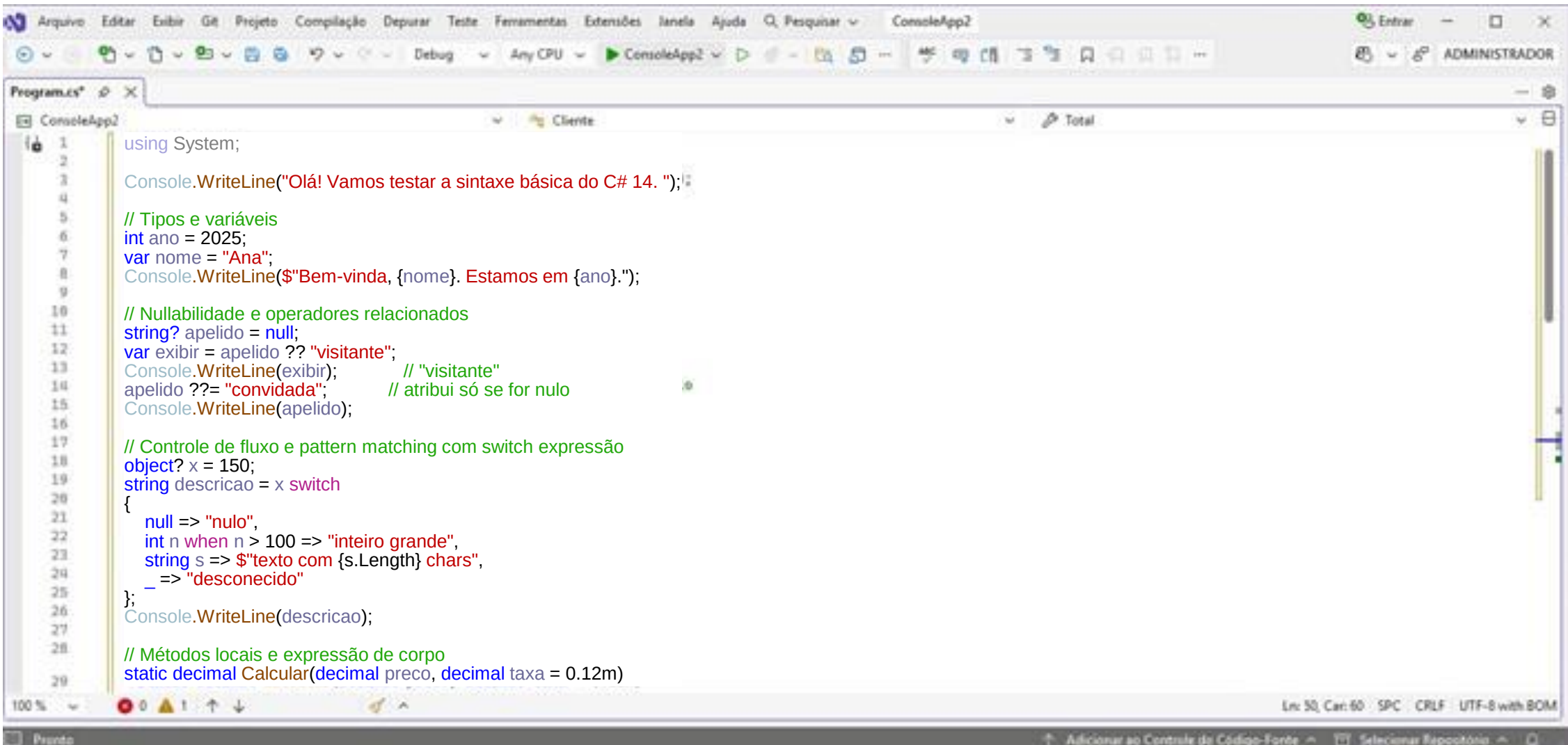
- Além dessas três edições principais da IDE, o ecossistema Visual Studio 2026 oferece versões complementares que muitas vezes aparecem na mesma página de download e acabam sendo confundidas com edições da IDE.
- Uma delas é o Visual Studio 2026 Insiders, que funciona como canal de pré-lançamento: builds mais novos, com recursos em teste, instalados lado a lado com versões estáveis e projetados para quem deseja experimentar novidades de desempenho, integração com GitHub Copilot e a interface atualizada antes de elas chegarem ao canal principal.
- Outra versão é o Visual Studio Team Explorer 2026, um cliente gratuito destinado a quem não programa, mas precisa interagir com Azure DevOps (itens de trabalho, repositórios, pipelines) sem instalar a IDE completa.
- Também aparecem no grupo de downloads do Visual Studio 2026 os Remote Tools e os Build Tools. Os Remote Tools para Visual Studio 2026 servem para depuração remota, testes e análise de desempenho em máquinas que não têm a IDE instalada, permitindo, por exemplo, anexar o depurador a um servidor ou dispositivo remoto usando apenas esse pacote auxiliar.

Visual Studio 2026

- Já os Build Tools para Visual Studio 2026 formam um conjunto específico para compilação via linha de comando ou sistemas de build (MSBuild, CMake etc.), sem o ambiente gráfico.
- Eles permitem compilar projetos ASP.NET, .NET, C++, Node.js, Python e outros workloads suportados pela IDE, o que é também condicionado a uma licença de Visual Studio quando utilizados em cenários proprietários, com exceção do uso na construção de dependências open source, de acordo com os termos de licença publicados.
- Tudo isso está ancorado na mesma base tecnológica do Visual Studio 2026, de modo que a escolha entre as versões é menos uma questão de capacidade de programar em X ou Y linguagem e mais uma decisão sobre requisitos organizacionais, governança e fluxo de trabalho em equipes de software.

Visual Studio 2026

■ Exemplo de código

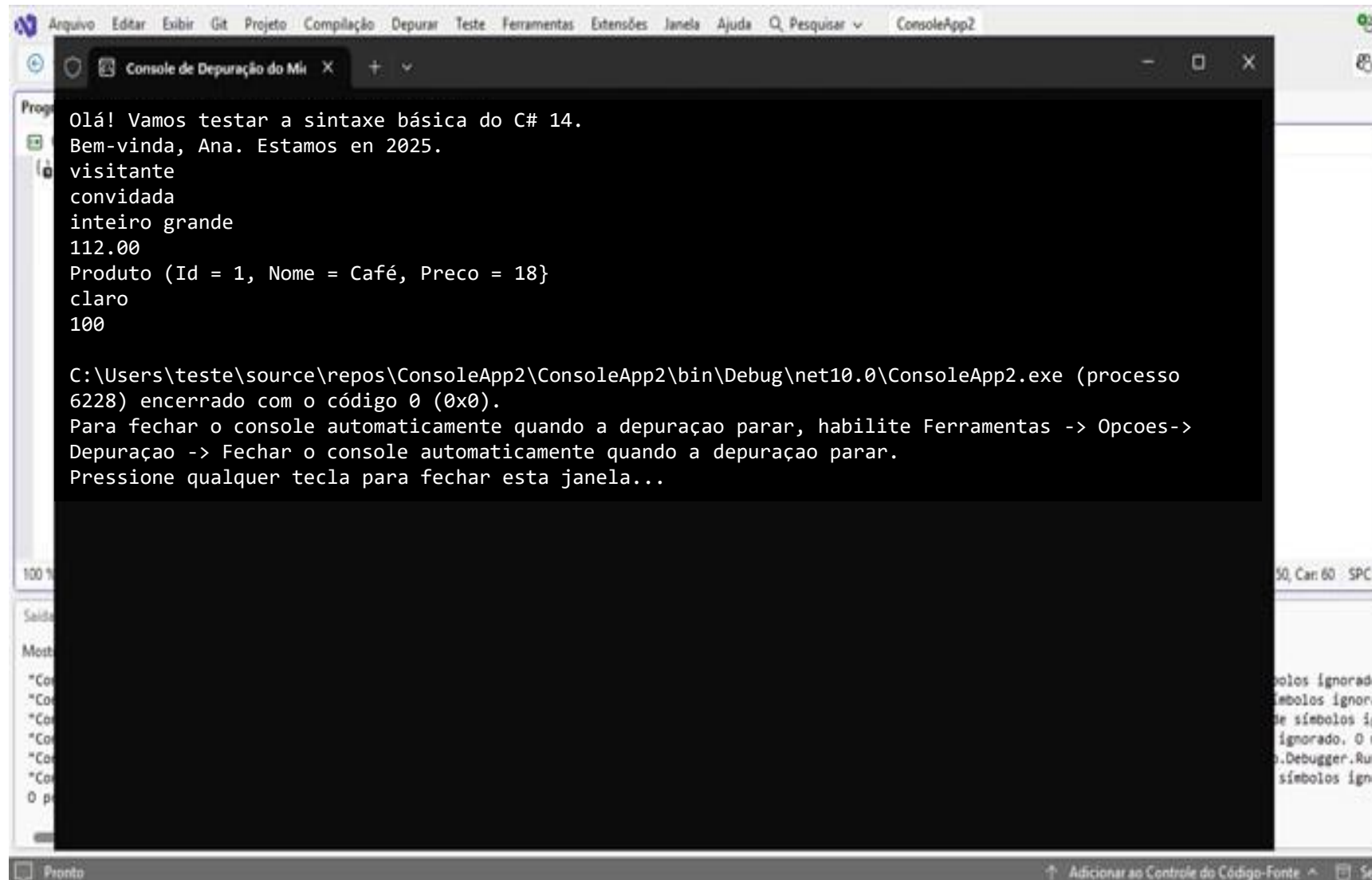


```
1 using System;
2
3 Console.WriteLine("Olá! Vamos testar a sintaxe básica do C# 14. ");
4
5 // Tipos e variáveis
6 int ano = 2025;
7 var nome = "Ana";
8 Console.WriteLine($"Bem-vinda, {nome}. Estamos em {ano}.");
9
10 // Nullabilidade e operadores relacionados
11 string? apelido = null;
12 var exibir = apelido ?? "visitante";
13 Console.WriteLine(exibir); // "visitante"
14 apelido ??= "convidada"; // atribui só se for nulo
15 Console.WriteLine(apelido);
16
17 // Controle de fluxo e pattern matching com switch expressão
18 object? x = 150;
19 string descricao = x switch
20 {
21     null => "nulo",
22     int n when n > 100 => "inteiro grande",
23     string s => $"texto com {s.Length} chars",
24     _ => "desconhecido"
25 };
26 Console.WriteLine(descricao);
27
28 // Métodos locais e expressão de corpo
29 static decimal Calcular(decimal preco, decimal taxa = 0.12m)
```

Fonte: autoria própria.

Visual Studio 2026

■ Saída no Console



The screenshot shows the Visual Studio 2026 interface with the 'Console de Depuração do Mi' window open. The console displays the output of a C# program. The text is as follows:

```
Olá! Vamos testar a sintaxe básica do C# 14.  
Bem-vinda, Ana. Estamos em 2025.  
visitante  
convidada  
inteiro grande  
112.00  
Produto (Id = 1, Nome = Café, Preço = 18}  
claro  
100  
  
C:\Users\teste\source\repos\ConsoleApp2\ConsoleApp2\bin\Debug\net10.0\ConsoleApp2.exe (processo  
6228) encerrado com o código 0 (0x0).  
Para fechar o console automaticamente quando a depuração parar, habilite Ferramentas -> Opcoes->  
Depuração -> Fechar o console automaticamente quando a depuração parar.  
Pressione qualquer tecla para fechar esta janela...
```

The console window is titled 'Console de Depuração do Mi' and has a tab for 'ConsoleApp2'. The status bar at the bottom shows 'Pronto' and 'Adicionar ao Controle do Código-Fonte'.

Fonte: autoria própria.

Visual Studio 2026

■ Exemplo de código

```
// -----  
// 1) IDENTIFICADORES CASE-SENSITIVE  
// -----  
  
// Aqui criamos duas variáveis com nomes quase iguais:  
// "aluno" (com 'a' minúsculo) e "Aluno" (com 'A' maiúsculo).  
// Em C#, isso representa dois identificadores diferentes.  
int aluno = 10;  
int Aluno = 20;  
  
Console.WriteLine("Exemplo de identificadores case-sensitive:");  
Console.WriteLine($"aluno = {aluno}"); // Usa a variável com 'a'  
minúsculo  
Console.WriteLine($"Aluno = {Aluno}"); // Usa a variável com 'A'  
maiúsculo  
Console.WriteLine();
```

Fonte: autoria própria.

Visual Studio 2026

■ Exemplo de código

```
// -----  
// 2) IF / ELSE IF / ELSE  
// -----  
// Vamos usar uma variável "nota" e classificar o desempenho do aluno.  
// Somente UM dos blocos será executado, de acordo com o valor de  
// "nota".  
int nota = 75; // Altere este valor (de 0 a 100) e veja o resultado  
mudar  
  
Console.WriteLine("Exemplo de if / else if / else com a variável  
'nota'.");  
Console.WriteLine($"Nota = {nota}");  
  
if (nota < 50)  
{  
    // Este bloco executa se a condição (nota < 50) for verdadeira  
    Console.WriteLine("Resultado: abaixo da média.");  
}  
else if (nota < 70)  
{  
    // Este bloco é testado se o anterior NÃO é verdadeiro  
    Console.WriteLine("Resultado: na média.");  
}  
else if (nota < 90)  
{  
    Console.WriteLine("Resultado: bom desempenho.");  
}  
else  
{  
    // Este bloco é o "caso padrão", quando nenhuma condição anterior  
    // é verdadeira  
    Console.WriteLine("Resultado: excelente desempenho!");  
}  
  
Console.WriteLine();
```

Fonte: autoria própria.

Visual Studio 2026

■ Exemplo de código

```
// -----  
// 3) SWITCH  
// -----  
// Agora usamos um "switch" para reagir a um comando simples.  
// valor possível em "comando": 'A' (Atacar), 'D' (Defender), 'F' (Fugir).  
char comando = 'A'; // Experimente mudar para 'D', 'F' ou outro caractere  
  
Console.WriteLine("Exemplo de switch com um comando simples (A, D ou F).");  
Console.WriteLine($"Comando = {comando}");  
  
switch (comando)  
{  
    case 'A':  
        // Se o comando for 'A', este bloco é executado  
        Console.WriteLine("Ação escolhida: Atacar.");  
        break; // Encerra o switch aqui  
  
    case 'D':  
        Console.WriteLine("Ação escolhida: Defender.");  
        break;  
  
    case 'F':  
        Console.WriteLine("Ação escolhida: Fugir.");  
        break;  
  
    default:  
        // Este bloco executa quando o valor de "comando" não é A, D nem F  
        Console.WriteLine("Ação desconhecida.");  
        break;  
}  
  
Console.WriteLine();
```

Visual Studio 2026

■ Exemplo de código

```
// -----  
// 4) LAÇO FOR  
// -----  
// O laço for é útil para repetir algo um número conhecido de  
// vezes.  
// Aqui vamos contar de 1 a 5.  
Console.WriteLine("Exemplo de laço for: contar de 1 até 5.");  
  
for (int i = 1; i <= 5; i++)  
{  
    // Dentro das chaves está o corpo do laço.  
    // A cada repetição, "i" é incrementado em 1 (i++)  
    Console.WriteLine($"i vale {i}");  
}  
  
Console.WriteLine();
```

Fonte: autoria própria.

Visual Studio 2026

■ Exemplo de código

```
// -----  
// 5) LAÇO WHILE  
// -----  
// O while repete ENQUANTO a condição for verdadeira.  
// Aqui fazemos uma contagem regressiva simples.  
Console.WriteLine("Exemplo de laço while: contagem regressiva a partir de  
3.");  
  
int contador = 3; // valor inicial, você pode alterar  
  
while (contador > 0)  
{  
    // Este bloco roda enquanto contador for maior que 0  
    Console.WriteLine($"Contador = {contador}");  
  
    // Diminui o valor de contador em 1 a cada repetição  
    contador--;  
}  
  
// Quando contador chega a 0, a condição (contador > 0) fica falsa  
// e o laço while termina  
Console.WriteLine("Fim da contagem regressiva.");  
Console.WriteLine();  
Console.WriteLine("Programa encerrado. Pressione ENTER para sair.");  
Console.ReadLine(); // Mantém a janela do console aberta até o usuário apertar  
ENTER
```

Fonte: autoria própria.

Ecossistema .NET

- C# é uma linguagem padronizada cuja implementação principal compila para a infraestrutura de linguagem comum;
- CLR é o runtime que executa o código intermediário (CIL), conforme definido pelo padrão ECMA-335;
- BCL é a coleção de tipos básicos que concretiza, do lado das bibliotecas, o modelo de tipos da CLI;
- .NET Framework e o .NET Core/.NET 5+ são pacotes que combinam versões específicas de CLR, BCL e bibliotecas adicionais;
- Visual Studio é o ambiente de desenvolvimento que integra compiladores, depuradores e designers para produzir assemblies destinados a essa infraestrutura.

Ecossistema .NET

- Runtime é o conjunto de componentes que fazem o programa ganhar vida na hora de executar. O arquivo do programa, sozinho, é só texto ou código gravado em disco.
 - Quando você pede para executar uma aplicação .NET, o runtime entra em ação: carrega o programa na memória, traduz o código para instruções que o processador entende, administra o uso de memória, cuida da execução passo a passo e encerra tudo de forma organizada quando o programa termina.
 - Em termos simples, o runtime é o ambiente que acompanha o programa enquanto ele está executando, garantindo que ele tenha tudo o que precisa para funcionar.
 - Biblioteca é um conjunto de funcionalidades prontas, empacotadas em arquivos de código que o desenvolvedor pode reaproveitar.
 - Em vez de cada programador ter que escrever do zero tudo o que um programa faz (por exemplo, mostrar texto na tela, acessar arquivos, conversar com a internet, trabalhar com datas e números grandes e assim por diante), essas tarefas comuns ficam reunidas em bibliotecas.

Ecossistema .NET

- O código da aplicação evoca as bibliotecas quando precisa de algo. Assim, o desenvolvedor escreve menos código, com menos chance de erro, apoiando-se em componentes já testados.
- SDK é a sigla para Software Development Kit, ou kit de desenvolvimento de software.
 - A ideia é parecida com um kit de ferramentas que você compra para realizar um tipo de trabalho específico.
 - Se você quer montar móveis, compra um kit com chaves de fenda, parafusos, manual e, às vezes, até gabaritos. No mundo da programação acontece algo semelhante: um SDK é um pacote que reúne tudo de que um desenvolvedor precisa para criar programas.
 - O dotnet CLI ocupa o lugar de ferramenta fundamental de linha de comando. A documentação oficial o descreve como uma interface multiplataforma para desenvolver, compilar, executar e publicar aplicações .NET, distribuída junto com o SDK do .NET; ao instalar o SDK, o desenvolvedor recebe a CLI, o runtime e bibliotecas de suporte em um único conjunto de ferramentas.

Interatividade

No site oficial da Microsoft, quais são as três edições do Visual Studio 2026?

- a) A edição Community, a edição Professional e a edição Enterprise.
- b) A edição Community, a edição Standard e a edição Ultimate.
- c) A edição Express, a edição Standard e a edição Enterprise.
- d) A edição Home, a edição Business e a edição Enterprise.
- e) A edição Starter, a edição Developer e a edição Enterprise.

Resposta

No site oficial da Microsoft, quais são as três edições do Visual Studio 2026?

- a) A edição Community, a edição Professional e a edição Enterprise.
- b) A edição Community, a edição Standard e a edição Ultimate.
- c) A edição Express, a edição Standard e a edição Enterprise.
- d) A edição Home, a edição Business e a edição Enterprise.
- e) A edição Starter, a edição Developer e a edição Enterprise.

Programação Orientada a Objetos

- A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento que modela o software como um conjunto de objetos, cada um combinando estado (atributos) e comportamento (métodos).
- A literatura clássica de engenharia de software descreve esse paradigma como uma estratégia para lidar com a complexidade de sistemas de grande porte por meio de um modelo de objetos que explora sistematicamente conceitos como abstração, encapsulamento, modularidade e hierarquia.
- A linguagem C# é definida como uma linguagem simples, moderna, orientada a objetos e segura quanto a tipos, inserida em um ambiente de execução (CLR) projetado para suportar diretamente essas construções.

POO – Classe

- Uma classe é uma descrição abstrata de um conjunto de coisas que têm características e comportamentos em comum.
- É como um modelo: nela o programador especifica quais dados existirão (os atributos ou campos) e quais ações poderão ser executadas (os métodos).
- A classe, por si só, funciona como um tipo de dado definido pelo programador, um molde lógico que organiza informações e operações relacionadas em um único bloco coerente.
- Entre esses métodos, em linguagens orientadas a objetos como C#, existe uma categoria especial chamada método construtor.
- O construtor é executado automaticamente quando um novo objeto da classe é criado e é usado para inicializar os atributos e estabelecer o estado inicial necessário para que esse objeto seja utilizado de maneira consistente.
- Em C#, ele tem o mesmo nome da classe, não declara tipo de retorno e pode receber parâmetros, permitindo que o código que cria o objeto forneça, já na construção, os valores ou as dependências indispensáveis ao seu uso.

POO – Objeto

- Um objeto, por sua vez, é uma realização concreta de uma classe durante a execução do programa.
- Quando o programa é executado e o código solicita a criação de algo com base em uma classe, o que surge na memória é um objeto.
- Esse objeto carrega valores específicos para os atributos definidos na classe e é capaz de executar os métodos que a classe descreve.

```
var carroDoAluno = new Carro("Azul", "Hatch", 2022);
```

```
class Carro
{
    public string Cor    { get; set; }
    public string Modelo { get; set; }
    public int  Ano      { get; set; }

    // Método construtor
    public Carro(string cor, string modelo, int ano)
    {
        Cor = cor;
        Modelo = modelo;
        Ano = ano;
    }

    public void Acelerar() { /* ... */ }
    public void Frear()   { /* ... */ }
}
```

Dot Operator

```
var carroDoAluno = new Carro("Azul", "Hatch", 2022);
```

- O que surge na memória é um objeto concreto, carroDoAluno, isto é, uma instância da classe Carro.
- Esse objeto possui valores específicos para os atributos definidos na classe (cor Azul, modelo Hatch, ano 2022) e passa a responder aos métodos declarados, de modo que chamadas como carroDoAluno.Acelerar(); e carroDoAluno.Frear(); utilizam o comportamento descrito na definição de Carro.
- Esse símbolo de ponto final é conhecido como dot operator e é o operador de acesso a membro.
- A documentação da linguagem o descreve como o token usado para acessar um membro de um namespace ou de um tipo, isto é, para navegar de um nome maior para um elemento mais específico: um namespace interno, um tipo dentro de um namespace ou um membro (campo, propriedade, método) de um tipo.

Dot Operator

- Em `System.Console.WriteLine("Olá")`, o ponto está sendo empregado em cadeia.
 - Primeiro, `System.Console` é um nome qualificado formado com `.` para indicar que a classe `Console` está dentro do namespace `System`.
 - Depois, em `Console.WriteLine`, o ponto é usado de novo, agora para acessar o método `WriteLine` definido nessa classe.
 - A própria documentação de operadores de acesso explica explicitamente esse uso: utilizar `.` para formar nomes qualificados de tipos em um namespace e para acessar membros de tipos, tanto estáticos quanto de instância.

POO – 4 pilares

- Uma convenção didática amplamente adotada em livros técnicos é descrever a POO a partir de quatro mecanismos centrais – encapsulamento, abstração, herança e polimorfismo, frequentemente chamados de “os quatro pilares” da orientação a objetos.
- Esses quatro mecanismos não são independentes: em conjunto, definem como uma aplicação em C# organiza seus tipos, limita o acesso a detalhes internos, reutiliza comportamentos entre classes e permite que diferentes objetos respondam de maneiras distintas a uma mesma operação.
- O primeiro pilar, o encapsulamento, diz respeito ao empacotamento de dados e operações que atuam sobre esses dados em uma unidade lógica, normalmente uma classe, isolando a representação interna e expondo apenas uma interface bem definida.

POO – 4 pilares

- Textos clássicos sobre POO com C++ e C# descrevem encapsulamento como a reunião de atributos e funções em uma mesma unidade, associada ao princípio de data hiding: o código externo não acessa diretamente o estado interno, mas interage por meio de métodos ou propriedades, o que permite controlar invariantes e restringir modificações indevidas.
- Em C#, isso se concretiza em decisões como declarar *campos privados*, expor propriedades com get/set controlados, usar modificadores de acesso (public, protected, internal, private) e definir métodos que representam operações conceitualmente relevantes sobre o objeto.
- Em um projeto moderno em C#, seja qual for a versão da linguagem, o encapsulamento continua sendo o mecanismo básico para garantir que mudanças na implementação interna de uma classe não obriguem a reescrita do código cliente, preservando a estabilidade das interfaces públicas – exatamente o tipo de benefício atribuído à orientação a objetos nos manuais acadêmicos.

POO – 4 pilares

- A abstração é o processo de selecionar os aspectos essenciais de uma entidade e ignorar detalhes contingentes, de modo que um tipo represente um conceito coerente do domínio de problema.
- Ela consiste em definir modelos que capturam propriedades conceitualmente relevantes, sem expor os mecanismos internos necessários para realizá-las.
- Em C#, classes e interfaces são os principais instrumentos de abstração: uma interface pode capturar um conjunto de operações que caracterizam, por exemplo, um repositório ou um serviço de notificação, sem impor como essas operações serão implementadas; classes concretas então realizam essa abstração em termos de campos, algoritmos específicos e recursos da plataforma.
- Do ponto de vista metodológico, a abstração orienta as etapas iniciais de análise e projeto: o desenvolvedor começa identificando as abstrações relevantes do domínio, mapeando-as para tipos, antes de tratar de detalhes de banco de dados, interface gráfica ou desempenho.

POO – 4 pilares

- A herança, terceiro pilar, é o mecanismo pelo qual uma classe (subclasse ou classe derivada) reaproveita e especializa a estrutura e o comportamento de outra (superclasse ou classe base).
- Todas as classes derivam, em última instância, de System.Object, e a linguagem oferece herança simples de classes combinada com implementação múltipla de interfaces, permitindo construir hierarquias em que tipos mais gerais capturam propriedades comuns, enquanto tipos mais específicos acrescentam comportamentos ou restringem usos.
- Em termos de projeto, a herança é usada em C# para modelar relações é-um (por exemplo, uma classe ContaPoupanca derivada de ContaBancaria) e para definir classes base abstratas que especificam operações obrigatórias para subclasses, favorecendo a reutilização de código e a definição de famílias de tipos com contratos compartilhados.

POO – 4 pilares

- O quarto pilar, o polimorfismo, é a capacidade de diferentes objetos responderem à mesma mensagem de formas específicas, preservando uma interface comum.
- O polimorfismo é descrito como a situação em que duas ou mais classes oferecem o mesmo conjunto de operações, mas com implementações distintas.
- O código cliente invoca a operação por meio de uma referência do tipo mais geral, e o sistema de despacho dinâmico escolhe, em tempo de execução, a implementação apropriada ao objeto concreto.
- Em C#, isso aparece na combinação de herança com métodos virtual e override, ou na programação voltada a interfaces.
- Esse mecanismo é fundamental para escrever código extensível: novas subclasses ou novas implementações de interfaces podem ser introduzidas sem alterar o código que depende apenas dos contratos abstratos.

POO – 4 pilares

- Tomados em conjunto, encapsulamento, abstração, herança e polimorfismo fornecem o arcabouço conceitual com que a literatura especializada descreve a POO em C#, e esse arcabouço continua válido nas versões mais recentes da linguagem e nas ferramentas modernas de desenvolvimento.
- O encapsulamento e a abstração determinam como as classes e as interfaces em C# 14 expressam modelos de domínio claros; a herança e o polimorfismo estruturam hierarquias de tipos e viabilizam arquiteturas baseadas em contratos que podem ser estendidos sem ruptura.
- Em um projeto desenvolvido em C# com suporte de IDEs atuais, aplicar esses quatro mecanismos de acordo com as recomendações continua sendo o caminho principal para obter sistemas mais compreensíveis, reutilizáveis e fáceis de evoluir, exatamente o objetivo que motivou o surgimento da orientação a objetos na engenharia de software.

Interatividade

Qual alternativa descreve corretamente como os quatro pilares da POO, em conjunto, orientam um projeto em C#?

- a) Eles atuam como mecanismos independentes: cada um resolve um tipo de problema sem afetar os demais.
- b) Eles definem como os tipos são organizados, como o acesso a detalhes internos é limitado, como comportamentos são reutilizados entre classes e como objetos diferentes podem responder de modo distinto à mesma operação.
- c) Eles servem apenas para a fase de codificação e não têm papel na análise e no desenho de sistemas.
- d) Eles substituem a necessidade de interfaces, pois a herança de classes em C# é múltipla.
- e) Eles têm validade apenas em versões antigas da linguagem e não se aplicam a C# 14.

Resposta

Qual alternativa descreve corretamente como os quatro pilares da POO, em conjunto, orientam um projeto em C#?

- a) Eles atuam como mecanismos independentes: cada um resolve um tipo de problema sem afetar os demais.
- b) Eles definem como os tipos são organizados, como o acesso a detalhes internos é limitado, como comportamentos são reutilizados entre classes e como objetos diferentes podem responder de modo distinto à mesma operação.
- c) Eles servem apenas para a fase de codificação e não têm papel na análise e no desenho de sistemas.
- d) Eles substituem a necessidade de interfaces, pois a herança de classes em C# é múltipla.
- e) Eles têm validade apenas em versões antigas da linguagem e não se aplicam a C# 14.

Encapsulamento na prática

- A implementação do encapsulamento costuma ser feita com propriedades (properties).
- Propriedades são membros da classe que funcionam como interfaces de acesso a campos privados: elas expõem metodologias de leitura (get) e escrita (set) controladas, muitas vezes substituindo os antigos métodos getters e setters manuais das linguagens orientadas a objetos clássicas.
- Apesar de, à primeira vista, criar uma propriedade pública com get/set parecer equivalente a ter um campo público, há uma diferença fundamental: a lógica interna do get/set permite validar, converter ou restringir o acesso aos dados antes de efetivar qualquer mudança.

Encapsulamento na prática

- Por exemplo, imagine uma classe que armazena internamente o horário em segundos (um `int` privado).
- Podemos expor esse dado por meio de uma propriedade pública `Time` do tipo `string`. O acessor `get` da propriedade poderia converter os segundos armazenados em um formato de hora legível “HH:MM:SS”, enquanto o acessor `set` aceitaria uma `string` nesse formato, converteria tal `string` para segundos e a armazenaria no campo privado.
- Durante esse processo, é possível incluir validações: ao receber uma `string`, o `set` pode checar se ela representa um horário válido – por exemplo, “12:30:45” é válido, mas “42:85:70” não – antes de atualizar o campo. Caso a validação falhe, a propriedade poderia lançar uma exceção ou ignorar a atribuição, impedindo que o objeto entre em um estado incorreto.
- Assim, mesmo para o código cliente que vê um atributo `Time` aparentemente público, internamente os acessores estão garantindo que qualquer interação ocorra de forma controlada e segura. Os dados privados permanecem protegidos – encapsulados – e consistentes dentro do objeto.

Exemplo

- A idade não pode ser negativa

```
public class Pessoa
{
    private int idade; // campo encapsulado

    public int Idade
    {
        get { return idade; }
        set
        {
            if (value < 0)
                throw new ArgumentOutOfRangeException("Idade
n\u00e3o pode ser negativa.");
            idade = value;
        }
    }
}
```

Exemplo

- A necessidade de garantir que um objeto nunca viole certas condições lógicas traz o conceito formal de invariante de classe.
- Um invariante de classe é uma condição que deve ser verdadeira para todo objeto válido daquela classe em qualquer momento observável externamente.
- O invariante deve valer antes e depois de cada chamada a métodos públicos do objeto, podendo ser quebrado temporariamente apenas durante a execução interna de um método (enquanto o objeto ajusta seu estado), desde que restaurado ao final da operação.
- No exemplo da classe Pessoa, a condição $\text{idade} \geq 0$ é um invariante: sempre que observamos um objeto Pessoa do mundo externo, esperamos que essa condição esteja mantida.
- Caso algum método interno precise alterar a idade (por exemplo, reduzir a idade em um ano – o que não faria sentido aqui, mas suponhamos um ajuste), ele poderia, durante parte de sua execução, trabalhar com um valor intermediário inválido, contanto que, ao término, a idade do objeto volte a satisfazer $\text{idade} \geq 0$.

Classe imutável

- Uma classe é considerada imutável quando suas instâncias, depois de criadas, não permitem alteração de estado – todas as informações contidas em cada objeto ficam fixas durante toda sua vida.
- Em vez de modificar o objeto, qualquer mudança conceitual é obtida criando-se um novo objeto com o novo valor desejado.
- Essa ideia, que tem raízes profundas na programação funcional, traz diversos benefícios: objetos imutáveis são intrinsecamente thread-safe (podem ser compartilhados entre threads sem risco de condições de corrida, já que nada muda), não precisam de cópias defensivas, e jamais apresentam “inconsistência temporária” – não há um estado intermediário inválido, pois o objeto simplesmente não muda após construído.
- Em outras palavras, se um objeto inicia sua vida obedecendo a todos os invariantes, ele permanecerá obedecendo, pois não há operações que o levem a um estado diferente.

Classe imutável

- As classes deveriam ser imutáveis a menos que haja uma razão muito boa para serem mutáveis.
- Ele sugere um conjunto de regras simples para criar uma classe imutável: não fornecer métodos que modifiquem o estado, declarar todos os campos como `private` e `final` (em C#, equivalentes seriam campos `private readonly`), e garantir acesso exclusivo a quaisquer componentes mutáveis internos (por exemplo, fazendo cópias defensivas).
- Além disso, todo objeto imutável deve ser completamente inicializado no fim do construtor, com todos os seus invariantes estabelecidos de saída.
- O C# 9 introduziu uma funcionalidade específica para facilitar a criação de objetos imutáveis: o `record`.
- `Records` são um tipo de referência especial em C# projetado para trabalhar bem com dados imutáveis (de apenas leitura).

Exemplo

```
public record Pessoa(string Nome, int Idade);
```

- Em uma única linha, criamos uma classe imutável Pessoa com duas propriedades (Nome e Idade).
- O compilador gerará uma propriedade public string Nome { get; init; } e public int Idade { get; init; }, além de um construtor que recebe Nome e Idade como parâmetros.
- As propriedades marcadas com init só podem ser atribuídas durante a construção do objeto (seja no construtor ou em um inicializador de objeto logo após), mas tornam-se somente-leitura após o objeto estar pronto.
- Assim, Pessoa p = new Pessoa("Ana", 30); cria um objeto cuja propriedade Idade vale 30, e nenhuma parte do código poderá alterá-la depois – não existe método set público. Qualquer tentativa de p.Idade = 31 resultaria em erro de compilação, garantindo imutabilidade.

Exemplo

- Se quisermos “modificar” a idade, a abordagem será criar um novo objeto derivado do existente, processo que o C# torna simples com a expressão with.
- Por exemplo: `Pessoa p2 = p with { Idade = 31 }`; cria um novo objeto Pessoa com mesmo Nome de p mas Idade alterada para 31, mantendo p original intocado.
- Esse é o chamado nondestructive mutation (mutação não destrutiva) – considerado o recurso mais útil dos records, pois automatiza o processo de copiar os dados imutáveis existentes para um novo objeto alterando apenas o necessário.
- Internamente, o compilador gera para isso um construtor de cópia protegido e implementa o operador with para que realize a cópia dos campos.
- Outra vantagem fornecida pelos records é que, por serem destinados a valores imutáveis, eles adotam por padrão a igualdade estrutural. Isso significa que dois objetos record do mesmo tipo são considerados iguais (`Equals` retorna `true`) se todos os seus campos públicos tiverem valores iguais.

Interatividade

Qual alternativa descreve corretamente a relação entre os invariantes de classe, o encapsulamento e os objetos imutáveis em C#?

- a) Um invariante de classe pode ser violado livremente, desde que o código cliente não perceba, e o encapsulamento não influencia essa garantia.
- b) Um invariante de classe deve ser verdadeiro apenas no início do programa; depois disso, cada método público pode deixá-lo em estado inválido.
- c) O encapsulamento permite centralizar e controlar mudanças no estado, ajudando a preservar invariantes; e objetos imutáveis simplificam essa preservação porque, uma vez construídos corretamente, não mudam mais de estado.
- d) Em C#, invariantes de classe são um recurso nativo da linguagem e são verificados automaticamente pelo compilador em tempo de execução.
- e) Records em C# só funcionam se todos os campos tiverem um set público, pois a ideia central é facilitar a mutabilidade.

Resposta

Qual alternativa descreve corretamente a relação entre os invariantes de classe, o encapsulamento e os objetos imutáveis em C#?

- a) Um invariante de classe pode ser violado livremente, desde que o código cliente não perceba, e o encapsulamento não influencia essa garantia.
- b) Um invariante de classe deve ser verdadeiro apenas no início do programa; depois disso, cada método público pode deixá-lo em estado inválido.
- c) O encapsulamento permite centralizar e controlar mudanças no estado, ajudando a preservar invariantes; e objetos imutáveis simplificam essa preservação porque, uma vez construídos corretamente, não mudam mais de estado.
- d) Em C#, invariantes de classe são um recurso nativo da linguagem e são verificados automaticamente pelo compilador em tempo de execução.
- e) Records em C# só funcionam se todos os campos tiverem um set público, pois a ideia central é facilitar a mutabilidade.

ATÉ A PRÓXIMA!